Programming Principles I

Week 7 · Class 14 · Quiz 4

Quiz 4: 25 minutes

Pen only. No laptop, no phone. Write your name and student ID first.

Programming Principles I

Week 7 · Class 14

Tables and matrices

Grids in functions · matrices · a table of points · the cells around a cell

```
int total(int g[][COLS], int rows);
```

What Tuesday gave us

1. `int t[2][3];` is a grid: 2 rows, 3 columns
2. `t[r][c]`: row first, then column; both start at 0
3. `{{4, 7, 1}, {9, 2, 6}}`: one pair of braces per row
4. Two loops: the outer one for the rows, the inner one for the columns
5. `#define ROWS 4` gives the size a name

Recap: what is on the screen?

PREDICT

One sum for each row.

What does it print?

```
int main(void) {
    int t[2][3] = {{4, 7, 1},
                   {9, 2, 6}};
    for (int r = 0; r < 2; r++) {
        int sum = 0;
        for (int c = 0; c < 3; c++) {
            sum += t[r][c];
        }
        printf("row %d: %d\n", r, sum);
    }
    return 0;
```

Recap: what is on the screen?

PREDICT

One sum for each row.

What does it print?

```c
int main(void) {
    int t[2][3] = {{4, 7, 1},
                   {9, 2, 6}};
    for (int r = 0; r < 2; r++) {
        int sum = 0;
        for (int c = 0; c < 3; c++) {
            sum += t[r][c];
        }
        printf("row %d: %d\n", r, sum);
    }
    return 0;
```

ANSWER

`row 0: 12` and `row 1: 17`

4 + 7 + 1 = 12. 9 + 2 + 6 = 17.

Today's roadmap

1. A grid goes to a **function**
2. **Matrices**: add, transpose, diagonal
3. A **table**: per row and per column
4. The cells **around** a cell
5. **Your turn**

> Part 4 is the heart of Project 2. Listen carefully there.

1 A grid goes to a function

The columns must be written

The rule for a grid parameter

> **`int g[][COLS]`: the rows can be empty, the columns cannot.**

The number of **rows** comes in as a parameter: `int rows`.

The number of **columns** is written in the brackets.

Call it with the name only: `print_grid(a, 3);`

A function that prints a grid

```c
#define COLS 4

void print_grid(int g[][COLS], int rows) {
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < COLS; c++) {
            printf("%3d", g[r][c]);
        }
        printf("\n");
    }
}
```

`g[][COLS]`: 4 columns, any number of rows.

`rows`: how many rows this grid has.

One function, two grids

```c
int main(void) {
    int a[3][COLS] = {{1, 2, 3, 4},
                      {5, 6, 7, 8},
                      {9, 10, 11, 12}};
    int b[2][COLS] = {{0, 0, 0, 0},
                      {1, 1, 1, 1}};
    print_grid(a, 3);
    printf("\n");
    print_grid(b, 2);
    return 0;
}
```

Output

```
 1  2  3  4
 5  6  7  8
 9 10 11 12

 0  0  0  0
 1  1  1  1
```

3 rows, then 2 rows.

Careful: both brackets empty

CAREFUL

Will this compile?

```
void print_first(int g[][], int rows) {
    for (int r = 0; r < rows; r++) {
        printf("%d\n", g[r][0]);
    }
}
```

No: gcc stops

```
void print_first(int g[][], int rows) {
    for (int r = 0; r < rows; r++) {
        printf("%d\n", g[r][0]);
    }
}
```

gcc says

```
error: array type
has incomplete
element type
'int[]'
```

It needs the columns: `g[][3]`.

Why C needs the columns

box in memory	0	1	2	3	4	5
cell	`[0][0]`	`[0][1]`	`[0][2]`	`[1][0]`	`[1][1]`	`[1][2]`

The rows sit one after another. Row 1 starts at box **3** because a row has **3** columns. Without that number C cannot find `g[1][0]`.

The function fills main's grid

```c
void times_table(int g[][COLS], int rows) {
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < COLS; c++) {
            g[r][c] = (r + 1) * (c + 1);
        }
    }
}

int main(void) {
    int t[ROWS][COLS];
    times_table(t, ROWS);
```

Output

```
 1  2  3  4
 2  4  6  8
 3  6  9 12
```

Not copied: the function writes into `t`.

A function that returns the total

```c
int total(int g[][COLS], int rows) {
    int sum = 0;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < COLS; c++) {
            sum += g[r][c];
        }
    }
    return sum;
}

int main(void) {
    int a[2][COLS] = {{1, 2, 3},
                      {4, 5, 6}};
    printf("total: %d\n", total(a, 2));
```

Output

```
total: 21
```

`sum` starts at 0, once.

2 Matrices

A grid of numbers, cell by cell

A matrix is a grid of numbers

Rows and columns, as in maths.

`m[i][j]`: row `i`, column `j`.

Two loops visit every cell: `i` outside, `j` inside.

A **square** matrix has as many rows as columns.

Add two matrices

```
int a[N][M] = {{1, 2, 3}, {4, 5, 6}};
int b[N][M] = {{10, 20, 30}, {40, 50, 60}};
int s[N][M];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        s[i][j] = a[i][j] + b[i][j];
    }
}
```

Output

```
 11  22  33
 44  55  66
```

Same place in a, b and s.

Transpose: rows become columns

```
int m[N][N] = {{1, 2, 3},
               {4, 5, 6},
               {7, 8, 9}};
int t[N][N];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        t[j][i] = m[i][j];
    }
}
```

Output

```
1  2  3
4  5  6
7  8  9

1  4  7
2  5  8
3  6  9
```

First `m`, then `t`. Row 0 of `m` is column 0 of `t`.

Predict: two diagonals

PREDICT

Add the cells of each diagonal.

What are `main_sum` and `other_sum`?

```
int m[N][N] = {{2, 0, 1},
               {3, 5, 4},
               {6, 7, 8}};
int main_sum = 0;
int other_sum = 0;
for (int i = 0; i < N; i++) {
    main_sum += m[i][i];
    other_sum += m[i][N - 1 - i];
}
```

Two diagonals

```
int m[N][N] = {{2, 0, 1},
               {3, 5, 4},
               {6, 7, 8}};
int main_sum = 0;
int other_sum = 0;
for (int i = 0; i < N; i++) {
    main_sum += m[i][i];
    other_sum += m[i][N - 1 - i];
}
```

Output

```
main diagonal: 15
other diagonal: 12
```

`m[i][i]`: 2 + 5 + 8.

`m[i][N - 1 - i]`: 1 + 5 + 6.

Which cells?

i	main: m[i][i]	other: m[i][N - 1 - i]
0	m[0][0] = 2	m[0][2] = 1
1	m[1][1] = 5	m[1][1] = 5
2	m[2][2] = 8	m[2][0] = 6

One loop is enough: on a diagonal the column follows from the row.

Careful: transpose in place

CAREFUL

Swap `m[i][j]` and `m[j][i]` for every `i` and `j`.

What is printed?

```
int m[N][N] = {{1, 2, 3},
               {4, 5, 6},
               {7, 8, 9}};
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        int temp = m[i][j];
        m[i][j] = m[j][i];
        m[j][i] = temp;
    }
}
```

Nothing changed

```
int m[N][N] = {{1, 2, 3},
               {4, 5, 6},
               {7, 8, 9}};
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        int temp = m[i][j];
        m[i][j] = m[j][i];
        m[j][i] = temp;
    }
}
```

Output

```
1  2  3
4  5  6
7  8  9
```

(0, 1) swaps 2 and 4.

(1, 0) swaps them back.

Swap each pair once

```
for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) {
        int temp = m[i][j];
        m[i][j] = m[j][i];
        m[j][i] = temp;
    }
}
```

Output

```
1  4  7
2  5  8
3  6  9
```

`j = i + 1`: only above the diagonal.

3 A table

One answer per row, or per column

Points: 4 players, 3 rounds

	round 1	round 2	round 3
player 1	7	9	4
player 2	5	6	8
player 3	9	3	6
player 4	4	8	10

`int pts[PLAYERS][ROUNDS]`: a row per player, a column per round.

Per row: each player's average

```
for (int p = 0; p < PLAYERS; p++) {
    int sum = 0;
    for (int r = 0; r < ROUNDS; r++) {
        sum += pts[p][r];
    }
    double avg = (double) sum / ROUNDS;
    printf("player %d: %.2f\n", p + 1, avg);
}
```

Output

```
player 1: 6.67
player 2: 6.33
player 3: 6.00
player 4: 7.33
```

Rows outside.

Per column: the best of each round

```
for (int r = 0; r < ROUNDS; r++) {
    int best = pts[0][r];
    for (int p = 1; p < PLAYERS; p++) {
        if (pts[p][r] > best) {
            best = pts[p][r];
        }
    }
    printf("round %d best: %d\n", r + 1, best);
}
```

Output

```
round 1 best: 9
round 2 best: 9
round 3 best: 10
```

Columns outside.

Start at the first row: `pts[0][r]`.

Which loop goes outside?

> **The outside loop is what you want one answer for.**

One answer **per row**: the row loop outside.

One answer **per column**: the column loop outside.

The index order never changes: `pts[p][r]`, row first.

4 The cells around a cell

Neighbours, and the edge of the grid

The 8 neighbours of (r, c)

	c - 1	c	c + 1
r - 1	(r-1, c-1)	(r-1, c)	(r-1, c+1)
r	(r, c-1)	**the cell**	(r, c+1)
r + 1	(r+1, c-1)	(r+1, c)	(r+1, c+1)

Across and diagonally. The cell itself is not its own neighbour.

Two small loops do it

`dr` and `dc` go from -1 to 1.

`r + dr`: the row above, this row, the row below.

`c + dc`: the column left, this column, right.

3 x 3 = 9 positions. Skip `dr == 0 && dc == 0`: 8 are left.

Question: how many neighbours?

QUESTION

In a 4 by 5 grid, how many neighbours does the corner cell (0, 0) have?

And a cell on the top edge, like (0, 2)?

Question: how many neighbours?

QUESTION

In a 4 by 5 grid, how many neighbours does the corner cell (0, 0) have?

And a cell on the top edge, like (0, 2)?

ANSWER

Corner: **3**. Edge: **5**. Middle: **8**.

The rest would be off the grid.

Careful: past the right edge

CAREFUL

`(0, 3)` is on the right edge. There is no column 4. What is printed?

```
int g[ROWS][COLS] = {{1, 2, 3, 4},
                     {5, 6, 7, 8},
                     {9, 10, 11, 12}};
int r = 0;
int c = 3;
printf("cell: %d\n", g[r][c]);
printf("right: %d\n", g[r][c + 1]);
```

A wrong neighbour, and no error

```
int g[ROWS][COLS] = {{1, 2, 3, 4},
                     {5, 6, 7, 8},
                     {9, 10, 11, 12}};
int r = 0;
int c = 3;
printf("cell: %d\n", g[r][c]);
printf("right: %d\n", g[r][c + 1]);
```

Output

```
cell: 4
right: 5
```

`g[0][4]` is the next box: `g[1][0]`.

No warning, even with `-Wall`.

is_inside: on the grid or not?

```c
#define ROWS 4
#define COLS 5

int is_inside(int r, int c) {
    return r >= 0 && r < ROWS && c >= 0 && c < COLS;
}

int main(void) {
    printf("%d\n", is_inside(0, 0));
    printf("%d\n", is_inside(3, 4));
    printf("%d\n", is_inside(-1, 2));
    printf("%d\n", is_inside(2, 5));
    return 0;
```

Output

```
1
1
0
0
```

Rows 0 to ROWS - 1, columns 0 to COLS - 1.

Count the neighbours of a cell

```
int count_neighbours(int r, int c) {
    int count = 0;
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;
            }
            if (is_inside(r + dr, c + dc)) {
                count++;
            }
        }
    }
    return count;
}
```

`continue`: skip the cell itself.

`is_inside`: drop positions off the grid.

Every cell of a 4 by 5 grid

```c
int main(void) {
    for (int r = 0; r < ROWS; r++) {
        for (int c = 0; c < COLS; c++) {
            printf("%2d", count_neighbours(r, c));
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
 3 5 5 5 3
 5 8 8 8 5
 5 8 8 8 5
 3 5 5 5 3
```

Corners 3, edges 5, middle 8.

A seating chart: 1 is taken

	0	1	2	3	4
0	1	0	0	1	1
1	0	1	0	0	1
2	1	1	0	1	0
3	0	0	0	1	1

`seats[2][1]` is 1: row 2, column 1 is taken.

Inside taken_around(s, r, c)

```
for (int dr = -1; dr <= 1; dr++) {
    for (int dc = -1; dc <= 1; dc++) {
        int nr = r + dr;
        int nc = c + dc;
        if (dr == 0 && dc == 0) {
            continue;
        }
        if (is_inside(nr, nc) && s[nr][nc] == 1) {
            count++;
        }
    }
```

nr, nc: the neighbour.

is_inside first, then read.

Predict: count by hand

	0	1	2	3	4
0	1	0	0	1	1
1	0	1	0	0	1
2	1	1	0	1	0
3	0	0	0	1	1

How many taken seats are around **(2, 2)**? Around the corner **(0, 0)**?

All four answers

```
printf("(0, 0): %d\n", taken_around(seats, 0, 0));
printf("(0, 2): %d\n", taken_around(seats, 0, 2));
printf("(2, 2): %d\n", taken_around(seats, 2, 2));
printf("(3, 4): %d\n", taken_around(seats, 3, 4));
```

It prints 1, 2, 4, 2. Around (2, 2): (1, 1), (2, 1), (2, 3), (3, 3) are taken. Around (0, 0): only (1, 1).

Check first, then read

```
is_inside(nr, nc) && s[nr][nc] == 1
```

`&&` stops at the first false part.

Off the grid: `is_inside` is 0, so `s[nr][nc]` is never read.

The other order reads outside the grid first. Never do that.

A grid of char: each row is a string

```
char map[ROWS][COLS + 1] = {"X..XX",
                            ".X..X",
                            "XX.X.",
                            "...XX"};
```

`COLS + 1`: 5 letters and `'\0'`.

`map[2][1]` is `'X'`: one character.

Print the map, count the X

```
printf("  01234\n");
for (int r = 0; r < ROWS; r++) {
    printf("%d ", r);
    for (int c = 0; c < COLS; c++) {
        printf("%c", map[r][c]);
        if (map[r][c] == 'X') {
            taken++;
        }
    }
    printf("\n");
}
printf("taken: %d\n", taken);
```

Output

```
  01234
0 X..XX
1 .X..X
2 XX.X.
3 ...XX
taken: 10
```

%c: one character.

5

Your turn

One function on paper

Your turn: count_value

YOUR TURN

Write a function `count_value`.

It gets a grid, its rows and a value `v`.

It returns how many cells are equal to `v`.

For `v` = 2 the answer is 3.

```
int a[3][COLS] = {{2, 7, 2, 0},
                  {5, 2, 9, 1},
                  {3, 4, 6, 8}};
```

count_value: one answer

```
int count_value(int g[][COLS], int rows, int v) {
    int count = 0;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < COLS; c++) {
            if (g[r][c] == v) {
                count++;
            }
        }
    }
    return count;
}
```

Output

```
3
0
```

count starts at 0, once.

Today in one slide

1. A grid parameter: `int g[][COLS]`, and the rows as `int rows`
2. The function works on the caller's grid: it is not copied
3. Matrices: `s[i][j] = a[i][j] + b[i][j]`; transpose `t[j][i] = m[i][j]`
4. Per row: rows outside. Per column: columns outside
5. Neighbours: `dr`, `dc` from -1 to 1; `is_inside` before you read

Before next week

1. **Knowledge check 14** on Blackboard: today's slides
2. **Project 2** (Treasure Hunt): due **Mon 9 November, 23:59**
3. **Midterm: Friday 30 October**, classes 1 to 14
4. Next class: **Tue 27 October**
5. Today's code: Blackboard, Week 7 module, **Class 14 code**

> Part 4 today is exactly what Project 2's clues need. Start the project this weekend.